
Lecture 7

Coding and Security

Outline

1. SQL Injection

2. Output Handling

3. Programming Languages Problem

- C/C++ Buffer Overflow
- Poisoned Null Byte / Null Byte Injection

4. Java Security

5. Java Security Demo

Programming Languages Security Threats

C/C++ Buffer Overflow

- C and C++ do not automatically check to see if we exceed an array bound.
 - The philosophy is that programmer does not pay for what he doesn't use. This makes the program faster.
 - If array bound checking is important it must be done by the programmer.
- A program that takes a user defined string and stores it in a char array (a buffer) can overwrite other arrays if the size of the string is not checked and the user provides an extra long string.
- This can cause the common **buffer overflow** attack.
- It can be prevented by explicitly checking the length of any input string.

C/C++ Memory Allocation

- C and C++ allocate memory using stack (at compile time) or heap (at run time).
- For instance:
 - To declare a variable on the stack:
`int numberPoints = 10;`
 - On the heap:
`int* ptr = malloc (10 * sizeof(int));`
- In general, stack overflow is more common than heap overflow

Original UNIX Password Checking

- Assume the memory is laid out sequentially as:

user	□	□	□	□	□	□	□	□
pwd	□	□	□	□	□	□	□	□
epwd	□	□	□	□	□	□	□	□
uepwd	□	□	□	□	□	□	□	□

Original UNIX Password Checking

➤ Assume the login process is:

1. Read in username and store it in `user`
2. Lookup the encrypted password and store it in `epwd`
3. Read in the user supplied password and store it in `pwd`
4. Encrypt the user supplied password, storing it in `uepwd`
5. Check if `uepwd` and `epwd` are the same.

Original UNIX Password Checking

- I want to log in as `root`, but don't know the `root` password.
 - The encrypted value of the root password is `!"£$%^&*`.
- I do know that password `abcdefgh` is encrypted to `stuvwxyz`.
- I enter `root` for the user, and `abcdefghstuvwxyz` for the password.
- After steps 1 and 2, the memory contains

user	r	o	o	t	□	□	□	□
pwd	□	□	□	□	□	□	□	□
epwd	!	"	£	\$	%	^	&	*
uepwd	□	□	□	□	□	□	□	□

Original UNIX Password Checking

- After step 3, the overlong password has overflowed from pwd to epwd, overwriting the value for root.

user	r o o t □ □ □ □
pwd	a b c d e f g h
epwd	s t u v w x y z
uepwd	□ □ □ □ □ □ □ □

- Step 4 then encrypts abcdefgh to produce stuvwxyz and epwd and uepwd are the same.

user	r o o t □ □ □ □
pwd	a b c d e f g h
epwd	s t u v w x y z
uepwd	s t u v w x y z

C/C++ Password Checking Demo

- To understand how a buffer overflow occurs, let's look at the following code, which performs a simple password check, and is susceptible to a buffer overflow attack:

- Online compiler:

<https://www.programiz.com/cpp-programming/online-compiler/>

Buffer Overflow Prevention

- Again, it can be prevented by explicitly checking the length of any input string.
- We might use managed buffers like `std::vector` or `std::string`
 - We lose our bounds-checking abilities as soon as we interface with C-style APIs that force us to pass `vector::data` or `string::c_str`.
- Coding Practices:
 - The best way to prevent buffer overflows is to use APIs that aren't vulnerable. In C++, we use managed buffers and strings rather than raw arrays and pointers.

Buffer Overflow Prevention

- `gets()` -> `fgets()` - read characters
- `strcpy()` -> `strncpy()` - copy content of the buffer
- `strcat()` -> `strncat()` - buffer concatenation
- `sprintf()` -> `snprintf()` - fill buffer with data of different types
- `(f)scanf()` - read from STDIN
- `getwd()` - return working directory
- `realpath()` - return absolute (full) path

Use safe equivalent functions, which check the buffers length, whenever it's possible.

C/C++ Poisoned null byte

- C and C++ use a null byte (`'\0'`) to terminate a string.
 - “Bob” is stored as `'B' 'o' 'b' '\0'`
- Other program languages (e.g., Python) provide an explicit length parameter for strings.
 - “Bob” is stored as `'3' 'B' 'o' 'b'`
- This mismatch means that we can use one language to explicitly include a **null character** in a string that is passed to a C or C++ program.

C/C++ Poisoned null byte

- For example, we might want to create a directory of images on an Apache server using PHP.
- The client is passed the name of an image file and a Python routine checks that the file either ends with **.jpg** or **.png**.
- The file is then passed to the server and processed.
- We can construct a file that contains attack code with a filename
 - `14 c r a c k . p h p \0 . j p g`
 - Python will accept it as an image file .
 - `c r a c k . p h p \0 . j p g` will be passed to the server.
 - The server will see the string `c r a c k . p h p` interpret it as a `.php` program file and run it.

Rules for Secure Coding

Use **POST** rather than **GET**

- Parameters are passed in the clear and can be changed by an attacker, who can resend the page.
- The browser can repeat a GET request whenever it wants.
 - If the user clicks the back button.
- This will have problems if the parameters change things on the server.
 - Making the same payment twice.

No Client-Side Security

- The attacker can send malicious information.
 - HTTP headers
 - Cookies
 - Hidden fields
 - Optional values.
- Don't trust the REFERER header in HTML requests.
 - It comes from the client and can be forged.
- Don't use client-side scripts for validation, authorisation, and authentication.

Generate a New Session ID After the User Logs On

- A session ID will be issued as soon as the user accesses a website.
- If it is used after the user logs in, then this is an escalation of privileges problem.
- It is easier for an attacker to get a session ID at the start of a session.

No Detailed Helpful Error Messages

- Remind: SQL Injection Attacks
- Attackers can provoke an error message in order to get information.

Take Care of Meta-Characters

- Identify every possible meta-character.
- Check meta-characters before passing data to subsystems.
 - A whitelist is better than a blacklist.
- Try to pass data separate from control information.
- Identify all sources of input to the application.
 - Not just data entered by the user.
 - Hidden fields
 - Check boxes and other GUI items.
- Once you have received information from the user, store it on the server.
 - Do not pass it back to the client in a hidden field.

Do Not Auto-Correct (Message) Invalid Data

- Just ask the user to enter the data again.
- Invalid input might be a sign of an attack.
- The attacker might anticipate the messaging and find a way round it.

Filter All Output Data

- Remind: Cross Site Scripting (XSS) Attacks.
- Make sure that information sent to the client's browser does not include attack code.

Quiz

6. What is a buffer overrun attack. What steps must be taken to prevent it.
7. What is a poisoned null byte attack. What steps must be taken to prevent it.